

A.P.E.X. Project: Phase 2 Official Task Assignments

To the Team:

I have built the core architecture, the FastAPI server, and the database schema. The repository is live on GitHub.

Because this is a 100-mark project for both Operating Systems and DBMS, I have divided the remaining work so that every person touches both an OS concept and a DBMS concept. You must complete your assigned file independently. Do not modify anyone else's file.

Project Links:

- GitHub Repository: <https://github.com/khushi-sharma2506/APEX>
- Setup: Clone the repo, run 'pip install -r requirements.txt'

1. Server Architecture & OS Concurrency (Khushi Sharma)

Assigned Files: src/main.py & src/database.py

Your Tasks:

- Initialize the FastAPI application and SQLite database structure.
- Build the core API endpoints.
- Implement OS-level Threading (ThreadPoolExecutor) so the server can handle 60 concurrent student requests without crashing.

2. Data Engineering & DB Transactions (Rohit Sharma)

Assigned Files: scripts/import_csv.py & data/mock_questions.csv

Your Tasks:

- Data Hunting: Find or create a CSV dataset of at least 500 mock multiple-choice questions. Format it cleanly with columns: Question, OptionA, OptionB, OptionC, OptionD, Answer, Difficulty, Topic.
- DB Ingestion: Write a Python script using the sqlite3 and csv modules to read your file and INSERT all 500 rows into the SQLite database.
- DBMS Concept: You must use SQL ACID Transactions (BEGIN, COMMIT) to ensure the data imports safely.

Status Check: I will review your work when you have successfully pushed the 500-question CSV and the working import script to GitHub.

3. OS Memory Manager (Roma Yadav)

Assigned File: src/cache.py

A.P.E.X. Project: Phase 2 Official Task Assignments

Your Tasks:

- OS Concept: Research and implement the OS LRU (Least Recently Used) Page Replacement algorithm.
- Code: Build a Python class LRUCache using collections.OrderedDict. Set a capacity (e.g., 100 items).
- Write a put() function to add questions to RAM, and a get() function to fetch them. If the cache hits 100 items, it must automatically evict the oldest item.

Status Check: I will review your work when you can run a local test proving the 1st item gets deleted when the 101st item is added.

4. Algorithm & Logic Engineer (Alok Goyal)

Assigned File: src/blueprint.py

Your Tasks:

- Research Component: You are building our 'Constraint-Aware Heuristic Algorithm'.
- Code: Create a function generate_paper(total_questions, hard_pct, med_pct, easy_pct).
- Write the logic to calculate exact quotas (e.g., 20% of 50 = 10 Hard questions).
- Write the Python sorting logic to loop through a dummy list of questions and return the exact mix requested. Handle edge cases.

Status Check: I will review your work when you can pass a fake list of 100 questions into your function and it successfully returns exactly a 20-question perfectly balanced test.

Workflow Rules

1. Work Independently: You do not need my server to be running to do your task. Test your logic using fake dummy data (fake lists/print statements) at the bottom of your file.
2. Deadlines: Have your isolated files working perfectly by the end of next week. After that, I will merge your classes into the main server.